

Enterprise Features & Architecture Operations Manual

A Comprehensive Reference to Placements, Operator Workflows, and Technical Background Lifecycles

Target System: Azam-Pnet / PNETLab Enterprise	Document Version: 2.5.0-LTS
Generated: September 19, 2026	Platform Scope: Core REST API + Canvas UI + Daemon Engine
System Daemon: azambasha-ops-api (:8088)	Security Classification: Enterprise Administrator Internal

1. Executive Architectural Taxonomy & Feature Placements

Azam-Pnet incorporates twenty-five modern enterprise features structured into three deliberate architectural planes: **Outside-the-Lab Canvas** (global system health, template management, backups, and hypervisor maintenance accessed from the main administrative navigation), **Inside-the-Lab Canvas** (in-workbench workflow tools, live console fixes, packet capture, AI assistance, and topology versioning available right inside the active topology view), and **Autonomous Kernel & Background Daemons** (self-healing services, crash recovery, socket repair, and network fabric acceleration operating headlessly 24/7).

Feature Name	Architectural Zone	Exact UI / System Location	Primary Trigger
1. Community Templates Marketplace	Outside Lab	Main Nav > Azam Features > Community Templates	One-Click Import
2. Live Hot-Node Profiler	Outside Lab	Main Nav > Azam Features > Health & Diagnostics	Real-Time / Kill
3. HTML5 Console Session Reset	Outside Lab	Main Nav > Azam Features > Console Fixer	Repair Button
4. Local & Cloud Lab Backups	Outside Lab	Main Nav > Azam Features > Cloud & Local Backup	Instant Backup
5. Fleet Multi-Node Cluster Monitor	Outside Lab	Main Nav > Azam Features > Cluster Nodes	Auto-Refresh (15s)
6. Lab Hardware Capacity Planner	Outside Lab	Main Nav > Azam Features > Capacity Planner	Interactive Sizer
7. Comprehensive Diagnostic Doctor	Outside Lab	Main Nav > Azam Features > System Doctor	Run Diagnostics
8. QCOW2 Disk Shrinker & Sparse Comp.	Outside Lab	Main Nav > Azam Features > Storage Optimizer	Batch Compress
9. Multi-Cloud Transit Overlay Bridge	Outside Lab	Main Nav > Azam Features > Cloud Transit	Connect Overlay
10. Automated SSL & WhatsApp Alerts	Outside Lab	Main Nav > Azam Features > Alerts & SSL	Deploy Cert / Test
11. Version Synchronizer & Installer	Outside Lab	Main Nav > Azam Features > Updates & Engine	Check & Update

12. Anti-Bootstorm Staggered Startup	Inside Canvas	Canvas Workbench > Top Toolbar > Anti-Bootstorm	Toolbar Button
13. Topology Git Version Control	Inside Canvas	Canvas Left Sidebar > Git VCS Icon > Drawer	Commit / Revert
14. In-Workbench Quick Console Healer	Inside Canvas	Canvas Workbench > Top Toolbar > Fix Console	Toolbar Button
15. Live Wireshark Packet Sniffer	Inside Canvas	Node Context Menu > Capture Interface > Stream	Right-Click Node
16. AI Topology & Lab Copilot	Inside Canvas	Canvas Bottom-Right > Floating AI Assistant	Chat & Analyze
17. Multi-Node Config Diff & Rollback	Inside Canvas	Canvas Workbench > Management Tools > Config Diff	Compare Configs
18. Automated Lab Exam Grader	Inside Canvas	Canvas Workbench > Lab Actions > Run Evaluation	Evaluate Lab
19. Ping Mesh & Traffic Generator	Inside Canvas	Canvas Workbench > Lab Testing > Traffic Injector	Generate Traffic
20. High-Res Diagram & SVG Exporter	Inside Canvas	Canvas Workbench > Export > Vector Topology	Download SVG
21. Interactive Canvas Accelerators	Inside Canvas	Canvas Navigation Controls (Minimap & Snap)	Always Active
22. 24/7 Self-Healing System Watchdog	Daemon Plane	systemd: azambasha-watchdog.service	Autonomous (60s)
23. MySQL Socket & Credential Healer	Daemon Plane	systemd & Cron: azambasha-fix-web-credentials	On Boot & Hourly
24. Soft-RoCE RXE & Jumbo MTU Engine	Kernel Plane	Kernel Module (ib_core, rdma_rxe) & udev	Boot & Net Hook
25. Scheduled Maintenance & TRIM Cron	Daemon Plane	cron.d: azambasha-maintenance-trim	Daily 03:00 UTC

2. Outside-the-Lab Features (Main Dashboard: /main/#/azam-features)

Features located in the primary management dashboard are accessible before loading any specific lab topology. They provide centralized hypervisor monitoring, repository synchronizations, fleet health oversight, and disaster recovery.

1. Community Templates Marketplace [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Community Templates tab (#pane-templates)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to the top administrative bar and click **Azam Features** (or open URL /main/#/azam-features).
- **Step 2:** Select the **Community Templates** tab located in the left navigation sidebar.
- **Step 3:** Browse or filter the community catalog (Cisco, Arista, Juniper, Fortinet, Linux, MikroTik).
- **Step 4:** Click the **Import Template** button next to the desired network operating system template.
- **Step 5:** Observe the live progress bar as image schemas and YAML definitions are verified.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end sends HTTP GET /api/azam/templates to azambasha-ops-api (port 8088).
- **Step 2:** API server reads /opt/unetlab/html/templates/ and parses device definitions.
- **Step 3:** When import is clicked, API issues POST /api/azam/templates/import with template ID.
- **Step 4:** Python backend invokes git clone --depth 1 or queries repository manifest schemas.
- **Step 5:** Script verifies template file syntax, generates corresponding /opt/unetlab/html/templates/{os}.yaml, sets permissions to www-data:www-data 0644, and flushes APC/OpCache.

2. Live Hot-Node Profiler & Task Manager [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Health & Diagnostics (#pane-health)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to /main/#/azam-features and select the **Health & Diagnostics** tab.
- **Step 2:** Scroll down to the **Hot-Node Live Performance Table**.
- **Step 3:** Inspect the real-time consumption table showing PID, Node Name, Lab Name, CPU %, and Memory %.
- **Step 4:** If a node is looping or hung, click the red **Kill Process** button in the Action column.
- **Step 5:** Confirm the termination modal to release hypervisor resources immediately.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Front-end polls GET /api/azam/perf/top every 4 seconds.
- **Step 2:** Backend runs ps aux --sort=-%cpu, filtering for QEMU (qemu-system-x86_64), IOL, and Dynamips.
- **Step 3:** Python script parses the process command line arguments to correlate PID to Lab UUID and Node ID via /opt/unetlab/tmp/ sockets.
- **Step 4:** On Kill trigger, POST /api/azam/perf-kill sends SIGTERM (15) to PID; if unresponsive after 2.5s, sends SIGKILL (9).
- **Step 5:** Kernel frees memory pages, closes virtual TAP interfaces, and records an audit entry in /var/log/azam-ops-api.log.

3. HTML5 Console Auto-Fixer & Session Cleaner [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Console Fixer (#pane-console)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the main dashboard, go to **Azam Features > System Doctor** or **Console Fixer**.
- **Step 2:** Click the primary button labeled **Run Console Full Repair**.
- **Step 3:** Wait 3-5 seconds while the system executes the recovery sequence.
- **Step 4:** Review the output modal showing Guacamole, Tomcat9, and Guacd restart statuses.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** API endpoint POST /api/azam/console-fix-full triggers azambasha-fix-web-credentials.sh --fix-guac.
- **Step 2:** System script kills orphaned websockify and stale Guacamole daemon worker threads.
- **Step 3:** Checks /etc/guacamole/guacamole.properties and MySQL Guacamole user authorization tables.
- **Step 4:** Executes systemctl restart guacd tomcat9 and verifies local socket listening on 127.0.0.1:4822.
- **Step 5:** Flushes browser websocket cookies and returns JSON status {success: true, services_reloaded: 3}.

4. Local Lab Archives & Multi-Cloud Backup [GUI: Outside Lab Canvas]

UI & System Placement: Main Dashboard Navigation > Azam Features > Cloud & Local Backups (#pane-cloud)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to **Azam Features > Cloud & Local Backups** tab.
- **Step 2:** To create an immediate local archive, click **Create Local Archive Now**.
- **Step 3:** To sync to remote cloud storage, configure AWS S3, Google Cloud Storage, or Mega credentials in the settings panel and click **Sync to Cloud**.
- **Step 4:** Download, inspect, or restore any snapshot from the **Available Local Lab Archives** table.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Request POST /api/azam/backups/local creates a timestamped tarball in /opt/unetlab/backups/labs/.
- **Step 2:** Background script bundles /opt/unetlab/labs/, calculating SHA256 hashes and compressing via pigz -p 4.
- **Step 3:** For cloud synchronization, azambasha-cloud-backup.sh loads Rclone / S3 credentials and executes rclone sync with AES-256 encryption.
- **Step 4:** Validates cloud checksums and logs transfer completion in /var/log/azam-backup.log.

5. Fleet Multi-Node Cluster Monitor [GUI: Outside Lab Canvas]

UI & System Placement: Main Dashboard Navigation > Azam Features > Cluster Fleet (#pane-cluster)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Go to **Azam Features > Cluster Fleet Monitor** tab.
- **Step 2:** View all connected PNETLab satellite worker nodes, CPU/RAM utilization, and cluster health.
- **Step 3:** Click **Add Satellite Node** and enter worker IP, SSH Port, and Shared Token.
- **Step 4:** Click **Balance Workload** to redistribute virtual nodes across cluster hypervisors.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Backend daemon azambasha-ops-api queries satellite worker endpoints every 15s via HTTPS/mTLS.
- **Step 2:** Aggregates remote CPU, RAM, and bridge interface states into centralized memory cache.
- **Step 3:** Evaluates node placement affinity rules against worker RAM capacity prior to new VM dispatch.
- **Step 4:** If a worker becomes unresponsive (missing 3 heartbeats), marks node as Degraded and triggers alert hook.

6. Lab Capacity & Hardware Sizing Planner [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Capacity Planner (#pane-capacity)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open **Azam Features > Capacity & Sizing Planner** tab.
- **Step 2:** Enter the estimated count of heavy appliances (e.g., Cisco XRv9k, Arista cEOS, Junos vMX, Linux servers).
- **Step 3:** The interactive calculator displays required vCPU, RAM, QCOW2 storage, and recommended KVM overcommit ratios.
- **Step 4:** Click **Apply Resource Limits** to enforce memory guardrails.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Calculator computes total theoretical footprint against actual available host hardware queried via `/proc/meminfo`.
- **Step 2:** Checks KSM (Kernel Samepage Merging) deduplication metrics via `/sys/kernel/mm/ksm/pages_sharing`.
- **Step 3:** Enforces safe limits by writing updated swap thresholds into `/etc/sysctl.d/99-pnetlab-perf.conf`.
- **Step 4:** Notifies administrator if requested deployment exceeds safe memory thresholds (over 85% host RAM).

7. Comprehensive Diagnostic Doctor [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > System Doctor (#pane-doctor)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to **Azam Features > System Doctor** tab.
- **Step 2:** Click **Run Full System Diagnostic**.
- **Step 3:** Observe 25+ automated checks spanning MySQL sockets, disk inodes, KVM permissions, bridge MTU, and DNS.
- **Step 4:** Click **Auto-Heal Detected Issues** to execute automated remediation.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Triggers execution of `/usr/local/bin/azambasha-doctor`.
- **Step 2:** Inspects MariaDB status via unix socket `/var/run/mysqld/mysqld.sock`.
- **Step 3:** Checks `/dev/kvm` permissions (0666), IP forwarding (`net.ipv4.ip_forward=1`), and bridge states.
- **Step 4:** If auto-heal is selected, executes repair routines, resets permissions on `/opt/unetlab/`, and returns diagnostic report.

8. QCOW2 Disk Shrinker & Sparse Compressor [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Storage Optimizer (#pane-storage)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Navigate to **Azam Features > Storage Optimizer** tab.
- **Step 2:** Select target lab or device image directory from the dropdown menu.
- **Step 3:** Click **Scan for Reclaimable Space**.
- **Step 4:** Click **Shrink & Compress Selected Images** and observe real-time reclaimed gigabytes.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Background worker scans `/opt/unetlab/addons/qemu/` and active lab storage.
- **Step 2:** Executes `qemu-img info` to calculate virtual size vs. actual allocated disk blocks.
- **Step 3:** Invokes `azambasha-heavy-node-optimizer.sh --shrink` which runs `qemu-img convert -c -O qcow2` with zlib compression.
- **Step 4:** Replaces original image atomically, updating sparse metadata and freeing unused hypervisor blocks.

9. Multi-Cloud Transit Overlay Bridge [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Cloud Transit (#pane-transit)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open **Azam Features > Cloud Transit** tab.
- **Step 2:** Select transit provider: **Tailscale**, **ZeroTier**, or **WireGuard**.
- **Step 3:** Paste network auth key or configuration token and click **Connect Transit Bridge**.
- **Step 4:** Interconnect local lab Management Cloud (pnet0/pnet1) directly to AWS VPC, Azure VNet, or home LAN.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** API executes azambasha-cloud-transit.sh with selected provider parameters.
- **Step 2:** Brings up virtual tun/tap interface (e.g. tailscale0 or wg0) in host kernel.
- **Step 3:** Configures iptables -t nat -A POSTROUTING -o <tun_interface> -j MASQUERADE and enables proxy ARP.
- **Step 4:** Bridges virtual lab nodes directly to the remote cloud subnet without requiring public IPs.

10. Automated SSL & WhatsApp Alerts [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Alerts & SSL (#pane-alerts)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Go to **Azam Features > Alerts & SSL** tab.
- **Step 2:** For SSL: enter fully qualified domain name (FQDN) and admin email, then click **Deploy Let's Encrypt SSL**.
- **Step 3:** For Alerts: enter Twilio Account SID, Auth Token, and WhatsApp recipient number.
- **Step 4:** Click **Send Test WhatsApp Alert** to verify real-time notification delivery.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** SSL process runs certbot certonly --standalone -d domain.com, binding temporarily to port 80.
- **Step 2:** Automatically updates Nginx configuration in /etc/nginx/sites-available/default with TLS certificates and HTTP/2.
- **Step 3:** Reloads Nginx and sets up automatic certbot renewal cron job in /etc/cron.d/certbot.
- **Step 4:** WhatsApp service stores encrypted credentials in /etc/pnetlab/alerts.conf and transmits JSON payload via Twilio REST API.

11. Version Synchronizer & Installer Engine [\[GUI: Outside Lab Canvas\]](#)

UI & System Placement: Main Dashboard Navigation > Azam Features > Updates & Engine (#pane-updates)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Go to **Azam Features > Updates & Engine** tab.
- **Step 2:** Review current installed version against latest upstream GitHub release.
- **Step 3:** Click **Check for Feature Updates**.
- **Step 4:** Click **Apply Updates & Re-link CLI Tools** to pull patches and regenerate system symlinks.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Queries GitHub release tags or local git repository using git rev-parse HEAD.
- **Step 2:** When upgrade is triggered, executes azambasha-install-azam-features.sh.
- **Step 3:** Installs and updates all 25 CLI symlinks in /usr/local/bin/ with executable permissions.
- **Step 4:** Restarts azambasha-ops-api.service and updates assets timestamp in browser cache.

3. Inside-the-Lab Canvas Features (Lab Workbench: /themes/default/)

Inside-the-lab features are embedded directly into the visual network topology canvas. They empower network engineers to execute live troubleshooting, staggered boot scheduling, version control commits, packet sniffing, and AI copilot queries without ever leaving the workbench.

12. Anti-Bootstorm Staggered Startup Engine [GUI: Inside Lab Canvas]

UI & System Placement: Lab Workbench Top Toolbar > **Anti-Bootstorm** button (#btn-azam-bootstorm)

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Open any lab topology inside the PNETLab canvas workspace.
- **Step 2:** In the top workbench navigation toolbar, locate the purple button labeled **Anti-Bootstorm** (icon: lightning bolt).
- **Step 3:** Click **Anti-Bootstorm** to open the Staggered Startup modal.
- **Step 4:** Select startup profile: **Conservative (15s delay)**, **Balanced (10s delay)**, or **Aggressive (5s delay)**.
- **Step 5:** Click **Start All Nodes Safely** and watch the nodes transition to running status sequentially.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Injected button triggers JavaScript function `azamLaunchBootstormModal()` which fetches lab node list.
- **Step 2:** JS posts to `POST /api/azam/bootstorm-start` passing `lab_path`, node IDs, and delay interval.
- **Step 3:** Backend runs `azambasha-bootstorm.py` which parses nodes by weight (firewalls & spine switches first).
- **Step 4:** Before booting each node, script monitors CPU utilization via `/proc/stat`; if load exceeds 80%, pauses execution until hypervisor cools down.
- **Step 5:** Boots nodes individually via `/opt/unetlab/wrappers/unl_wrapper -a start -i {node_id}` preventing CPU spikes and OOM panics.

13. Topology Git Version Control System (VCS) [GUI: Inside Lab Canvas]

UI & System Placement: Lab Workbench Left Sidebar > **Git VCS** icon (#btn-azam-git-vcs) > Slide-Over Drawer

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the lab canvas, look at the left-hand vertical toolbar.
- **Step 2:** Click the **Git Version Control** icon (branch icon, #btn-azam-git-vcs).
- **Step 3:** A modern slide-over drawer appears showing commit history, author, and timestamp.
- **Step 4:** To save current topology state: enter a commit message (e.g. 'Added OSPF Area 0') and click **Commit Topology State**.
- **Step 5:** To rollback: click **Restore** next to any historical commit; the canvas automatically refreshes to that state.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Drawer initialization triggers `GET /api/azam/topology-log?lab={lab_name}`.
- **Step 2:** Backend verifies that the lab directory `/opt/unetlab/labs/<lab>` is a Git repository; if not, performs automatic git init.
- **Step 3:** When commit is triggered, `POST /api/azam/topology-commit` stages the `.unl XML` file and runs `git commit -m 'message'`.
- **Step 4:** When diff is requested, `git diff HEAD~1` compares XML node and connection coordinate tags.
- **Step 5:** On rollback, `git checkout <hash> -- <lab.unl>` restores file and sends a reload event to the browser canvas.

14. In-Workbench Quick Console Healer [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: *Lab Workbench Top Toolbar > **Fix Console** button (#btn-azam-fix-console)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** While working inside the lab, if clicking a node yields a blank screen or 'Connection Refused', do not leave the canvas.
- **Step 2:** Click the green button in the top toolbar labeled **Fix Console** (icon: wrench).
- **Step 3:** A confirmation notification appears: 'Restarting HTML5 Console Services...'
- **Step 4:** Wait 2 seconds until the success toast appears ('Console services recovered successfully').
- **Step 5:** Re-click the node to open the working HTML5 terminal immediately.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Button executes `azamFixConsoleNow()` which sends an asynchronous POST `/api/azam/console-fix-full`.
- **Step 2:** API server invokes `azambasha-fix-web-credentials.sh` targeting Guacamole and Tomcat daemon sockets.
- **Step 3:** Flushes stuck client sessions in Guacamole memory cache and verifies port 4822 binds properly.
- **Step 4:** Resets browser terminal session iframe tokens without requiring a full page refresh.

15. Live Wireshark Packet Sniffer & Telemetry [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: *Lab Workbench Node / Link Context Menu > **Live Capture (Wireshark)***

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Right-click on any device node or connection link inside the canvas.
- **Step 2:** Select **Live Capture (Wireshark)** and pick the interface (e.g. GigabitEthernet0/0).
- **Step 3:** The Live Sniffer drawer opens at the bottom with real-time packet protocol decode.
- **Step 4:** Click **Download PCAP** or **Launch Wireshark App** to analyze packets locally.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Request queries PNETLab virtual bridge interface corresponding to selected node and port (e.g., `vnetX_Y`).
- **Step 2:** Host spawns `tcpdump -i vnetX_Y -U -w -` streaming raw pcap bytes over HTTP chunked transfer or websocket.
- **Step 3:** Script pipes output to web-based packet parser or native Wireshark wrapper via SSH tunnel.
- **Step 4:** Terminates `tcpdump` process cleanly upon closing the capture modal.

16. AI Lab Copilot & Assistant [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: *Lab Workbench > Floating Action Widget (Bottom-Right) > **AI Copilot Modal***

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Click the floating **AI Assistant** widget at the bottom-right corner of the canvas.
- **Step 2:** Type a query such as 'Why isn't OSPF forming neighbor adjacency between R1 and R2?' or 'Generate BGP EVPN config for Spine-Leaf'.
- **Step 3:** Review the generated configuration, troubleshooting steps, and topology analysis.
- **Step 4:** Click **Copy Configuration** to paste directly into device consoles.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Client extracts current topology structure (nodes, images, links, subnets) from canvas DOM and XML.
- **Step 2:** Sends sanitized context payload to backend AI proxy endpoint POST `/api/azam/ai-copilot`.
- **Step 3:** AI Copilot engine analyzes network architecture against vendor syntax guides (Cisco, Arista, Juniper).
- **Step 4:** Streams structured markdown responses and executable CLI snippets back to the canvas modal.

17. Multi-Node Config Diff & Rollback [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: Lab Workbench > Canvas Management Menu > Config Diff Engine

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the lab, click **Management Tools** in the canvas sidebar > **Config Diff Engine**.
- **Step 2:** Select nodes to compare or choose between two historical configuration snapshots.
- **Step 3:** The split-screen diff viewer highlights added lines in green, removed lines in red, and modified syntax in yellow.
- **Step 4:** Click **Rollback Node Config** to push previous baseline syntax back to the device.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Tool queries /opt/unetlab/tmp/{lab_id}/{node_id}/startup-config and running snapshots.
- **Step 2:** Python difflib compares normalized configuration text, ignoring non-functional whitespace.
- **Step 3:** Renders unified diff payload and returns structured JSON to client-side CodeMirror diff editor.
- **Step 4:** On rollback, writes chosen config directly to node nvram/flash storage via unl wrapper.

18. Automated Lab Exam Grader [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: Lab Workbench Top Bar > Lab Actions > Run Exam Evaluation

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Click **Lab Actions** in the canvas top menu > **Run Lab Assessment / Exam Grader**.
- **Step 2:** Select test rubric (e.g. CCNA Routing, BGP Multi-Homing, OSPF Area Verification).
- **Step 3:** Click **Execute Validation Tests**.
- **Step 4:** View real-time checklist: green ticks for passed objectives, red marks for failed requirements, and final score percentage.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Backend orchestrator initiates non-intrusive automated Telnet/SSH probes to target node console ports.
- **Step 2:** Executes verification commands (e.g. show ip route, show ip bgp summary).
- **Step 3:** Regex parsing engine validates prefix reachability, next-hop IP, and protocol neighbor states.
- **Step 4:** Generates score card and saves PDF/JSON certificate in /opt/unetlab/labs/{lab}/results/.

19. Ping Mesh & Traffic Generator Engine [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: Lab Workbench Top Bar > Lab Testing > Ping Mesh & Traffic

Operator Workflow (Step-by-Step How-To):

- **Step 1:** In the canvas menu, select **Lab Testing > Ping Mesh & Traffic Generator**.
- **Step 2:** Select participating nodes or choose 'Full Mesh All Running Nodes'.
- **Step 3:** Set packet size (64 to 9000 bytes) and rate (pps).
- **Step 4:** Click **Start Mesh Test** to view latency matrix heatmap, packet loss, and jitter.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Orchestrator dispatches lightweight ICMP/UDP echo requests across all node bridge endpoints.
- **Step 2:** Collects round-trip time (RTT) telemetry and calculates minimum, average, and maximum latency.
- **Step 3:** Renders interactive matrix heatmap in the canvas interface.
- **Step 4:** Detects path MTU blackholes if packets above 1500 bytes drop unexpectedly.

20. High-Resolution Lab Diagram Exporter [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: Lab Workbench Top Bar > Export > Export Topology Diagram

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Inside the canvas, click **Export** in the top-right toolbar > **Export Topology Vector/PNG**.
- **Step 2:** Choose output format: **Scalable Vector Graphics (SVG)** or **High-Res PNG (300 DPI)**.
- **Step 3:** Toggle options: Include IP labels, Include interface names, Watermark with company logo.
- **Step 4:** Click **Download Image File** for presentation-ready architecture diagrams.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** JavaScript reads SVG DOM elements representing nodes, custom shapes, text labels, and link paths.
- **Step 2:** Normalizes viewBox coordinates, converts external image icons into inline base64 data URIs.
- **Step 3:** Applies CSS styling and embeds vector fonts directly into the standalone SVG document.
- **Step 4:** Triggers instant browser download without round-tripping to server, preserving client privacy.

21. Interactive Canvas Accelerators & Minimap [\[GUI: Inside Lab Canvas\]](#)

UI & System Placement: Lab Workbench Canvas > Bottom-Left Minimap & Keyboard Accelerators

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Locate the interactive **Minimap Overview** at the bottom-left of the canvas.
- **Step 2:** Drag the viewport indicator in the minimap to pan smoothly across large topologies with 50+ nodes.
- **Step 3:** Select multiple nodes and press **Shift + S** to snap to grid or **Shift + A** for automatic alignment.
- **Step 4:** Use mouse wheel for continuous zoom from 25% overview to 200% detail view.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Canvas rendering engine creates a secondary hardware-accelerated HTML5 canvas element for the minimap.
- **Step 2:** Listens to canvas pan/zoom transform matrices and recalculates viewport bounding box in real-time.
- **Step 3:** Snap-to-grid algorithm calculates modulo-20 coordinate snapping for node drag events.
- **Step 4:** Persists updated coordinates directly to the underlying .unl XML file upon mouse release.

4. Background Daemons & Kernel Infrastructure

These core features operate autonomously beneath the graphical interface. They maintain hypervisor stability, repair broken MySQL sockets, enable hardware-accelerated networking, and perform scheduled system cleanup without requiring user intervention.

22. 24/7 Self-Healing System Watchdog Daemon [\[Background Daemon Plane\]](#)

UI & System Placement: *Linux Systemd Service > azambasha-watchdog.service (Autonomous 24/7)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** The watchdog runs autonomously as a Linux systemd service: `azambasha-watchdog.service`.
- **Step 2:** To inspect live status, open an SSH terminal and run: `systemctl status azambasha-watchdog`.
- **Step 3:** To view event logs: `tail -f /var/log/azambasha-watchdog.log`.
- **Step 4:** No manual interaction is needed; the service auto-starts on boot and runs continuously.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Daemon wakes up every 60 seconds and evaluates host health metrics.
- **Step 2:** Inspects Apache2, Nginx, Guacd, and Tomcat9 process IDs; if any service is down, triggers automated restart.
- **Step 3:** Monitors available system RAM; if free RAM drops below 5%, identifies hung QEMU processes and sends alerts.
- **Step 4:** Cleans up zombie processes and stale temporary lock files in `/tmp` to prevent hypervisor starvation.

23. MySQL Socket & Credential Auto-Healer [\[Background Daemon Plane\]](#)

UI & System Placement: *Systemd Hook & Cron > /usr/local/bin/azambasha-fix-web-credentials*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** The healer runs automatically during boot, hourly via cron, and upon admin request.
- **Step 2:** To trigger manual repair from CLI, execute: `azambasha-fix-web-credentials`.
- **Step 3:** The script inspects database user passwords, table permissions, and socket symlinks.
- **Step 4:** Confirms with output message: `[OK] Web credentials and MySQL socket successfully verified.`

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Checks if MariaDB unix socket exists at `/var/run/mysqld/mysqld.sock`; if missing, creates required directory and symlink.
- **Step 2:** Connects to database and verifies pnetlab user credentials match `/opt/unetlab/html/includes/config.php`.
- **Step 3:** Synchronizes Guacamole database credentials in `/etc/guacamole/guacamole.properties`.
- **Step 4:** Flushes MariaDB privileges (`FLUSH PRIVILEGES;`) and verifies read/write integrity.

24. Soft-RoCE RXE & MTU 9000 Network Accelerator [\[Kernel Infrastructure Plane\]](#)

UI & System Placement: *Linux Kernel Module & Network Configuration (/etc/modprobe.d/, udev)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Activated during initial setup via `azambasha-os-prerequisites.sh`.
- **Step 2:** To inspect RDMA status, run: `rdma link` or `ibv_devices`.
- **Step 3:** To verify Jumbo Frame MTU across bridge interfaces: `ip link show | grep mtu`.
- **Step 4:** All virtual bridges (`pnet0` through `pnet9`) operate with MTU 9000 for high-throughput packet emulation.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Loads Linux kernel modules rdma_rxe and ib_core into kernel memory.
- **Step 2:** Binds software RoCE (RDMA over Converged Ethernet) devices to physical NIC interfaces.
- **Step 3:** Configures udev rules and /etc/network/interfaces setting bridge MTU to 9000.
- **Step 4:** Eliminates CPU packet fragmentation overhead for high-speed emulated links (e.g. 10G/40G Arista and Cisco data center fabrics).

25. Scheduled Maintenance & Storage TRIM Engine [\[Background Daemon Plane\]](#)

UI & System Placement: *System Cron > /etc/cron.d/azambasha-maintenance (Nightly 03:00 UTC)*

Operator Workflow (Step-by-Step How-To):

- **Step 1:** Scheduled automatically via cron: /etc/cron.d/azambasha-maintenance.
- **Step 2:** Executes nightly at 03:00 UTC without disrupting running user labs.
- **Step 3:** To run on demand from CLI: azambasha-heavy-node-optimizer.sh --maintenance.
- **Step 4:** Check maintenance log at: /var/log/azambasha-maintenance.log.

Internal Mechanics & Background Execution Lifecycle:

- **Step 1:** Executes fstrim -av to issue TRIM commands to underlying SSD/NVMe storage, recovering deleted blocks.
- **Step 2:** Clears Linux dentry and inode pagecache buffers via sysctl vm.drop_caches=3 if system is idle.
- **Step 3:** Truncates log files in /opt/unetlab/data/Logs/ older than 14 days and compresses historical archives.
- **Step 4:** Empties orphaned sockets in /opt/unetlab/tmp/ left behind by abnormally terminated nodes.

5. Enterprise CLI Command-Line Reference

In addition to the visual web interface, all Azam-Pnet enterprise features are accessible directly via SSH terminal through globally linked utilities in `/usr/local/bin/`. The table below outlines all available CLI commands.

Command / Utility	Target Feature / Scope	Typical Invocation / Options
azambasha-ops-api	REST API Daemon Backend	<code>systemctl start azambasha-ops-api</code>
azambasha-bootstorm	Anti-Bootstorm Engine	<code>azambasha-bootstorm --lab /opt/unetlab/labs/lab.unl</code>
azambasha-doctor	Diagnostic Health Suite	<code>azambasha-doctor --full --auto-heal</code>
azambasha-fix-web-credentials	MySQL & Guac Socket Fixer	<code>azambasha-fix-web-credentials --fix-guac</code>
azambasha-heavy-node-optimizer	QCOW2 Disk & RAM Optimizer	<code>azambasha-heavy-node-optimizer --shrink</code>
azambasha-cloud-backup	Local & Cloud Backup Engine	<code>azambasha-cloud-backup --local --cloud s3</code>
azambasha-cloud-transit	Cloud Transit Overlay Bridge	<code>azambasha-cloud-transit --tailscale --connect</code>
azambasha-watchdog	24/7 Autonomous Watchdog	<code>azambasha-watchdog --daemon</code>
azambasha-install-azam-features	Symlink & Package Installer	<code>azambasha-install-azam-features.sh</code>
azambasha-fix-export-and-apt	Ubuntu Package Repository Fix	<code>azambasha-fix-export-and-apt.sh</code>
azambasha-fix-network-boot	Bridge & TAP Network Boot Fix	<code>azambasha-fix-network-boot.sh</code>

Documentation Summary & Compliance Note:

This manual encompasses all enterprise features integrated into the Azam-Pnet platform. All API endpoints, UI placements, and background scripts adhere to the platform's non-destructive design architecture. Stale websockets, hypervisor memory thresholds, and disk structures are guarded by automated checks to guarantee continuous lab uptime.